UNIT – V

BRANCH AND BOUND METHOD

Branch and bound method : Least Cost (LC) search – the 15-puzzle problem – control abstractions for LC-Search – Bounding – FIFO Branch-and-Bound - 0/1 Knapsack problem – traveling salesman problem. Introduction to NP-Hard and NP-Completeness.

SubTopics:

LC Search
- Control abstractions for LC Search
- Eg:15 puzzle problem

Bounding
- Eg:Job scheduling problem
- FIFO Barnch & Bound
- LC Branch & Bound

0/1 Knapsack problem
- FIFO Barnch & Bound solution
- LC Branch & Bound solution

5.1. INTRODUCTION

The design technique known as **branch and bound** is very similar to backtracking(seen in unit 4) in that it searches a tree model of the solution space and is applicable to a wide variety of discrete combinatorial problems.

Each node in the combinatorial tree generated in the last Unit defines a problem state. All paths from the root to other nodes define the **state space** of the problem.

Solution states are those problem states 's' for which the path from the root to 's' defines a tuple in the solution space. The leaf nodes in the combinatorial tree are the solution states.

Answer states are those solution states 's' for which the path from the root to 's' defines a tuple that is a member of the set of solutions (i.e.,it satisfies the implicit constraints) of the problem.

The tree organization of the solution space is referred to as the **state space tree.**

A node which has been generated and all of whose children have not yet been generated is called a **live node.**

The **live node** whose children are currently being generated is called the E-node (node being expanded).

A **dead node** is a generated node, which is not to be expanded further or all of whose

children have been generated.

Bounding functions are used to kill live nodes without generating all their children.

Depth first node generation with bounding function is called backtracking. State generation methods in which the E-node remains the E-node until it is dead lead to **branch-and-bound method.**

The term branch-and-bound refers to all state space search methods in which all children of the E-node are generated before any other live node can become the E-node.

In branch-and-bound terminology breadth first search(BFS)- like state space search will be called FIFO (First In First Output) search as the list of live nodes is a first -in-first -out list(or queue).

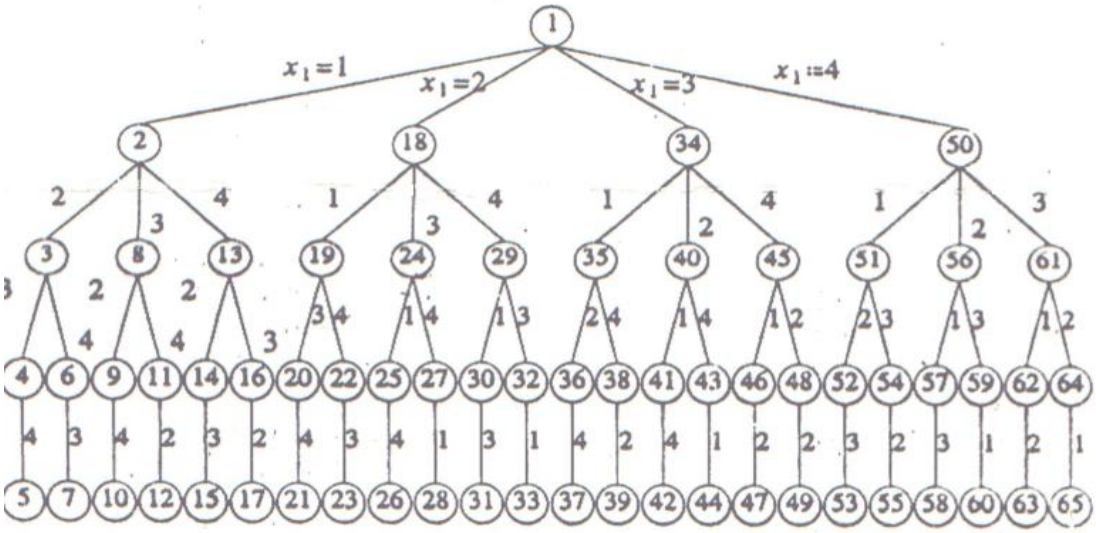
A **D-search** (depth search) state space search will be called LIFO(Last In First Out)search, as the list of live nodes is a list-in-first-out list (or stack).

Bounding functions are used to help avoid the generation of subtrees that do not contain an answer node.

The branch-and-bound algorithms search a tree model of the solution space to get the solution. However, this type of algorithms is oriented more toward optimization. An algorithm of this type specifies a real -valued cost function for each of the nodes that appear in the search tree.

Usually, the goal here is to find a configuration for which the cost function is minimized. the branch-and-bound algorithms are rarely simple. they tend to be quite complicated in many cases.

Example 8.1[4-queens] Let us see how a FIFO branch-and-bound algorithm would search the state space tree (figure 7.2) for the 4-queens problem.



Initially, there is only one live node, node1. This represents the case in which no queen has been placed on the chessboard. This node becomes the E-node.

It is expanded and its children,nodes2,18,34,and 50,are generated.

These nodes represent a chessboard with queen1 in row 1and columns 1, 2, 3, and 4 respectively.

The only live nodes 2, 18, 34, and 50.If the nodes are generated in this order, then the next E-node is node 2.

It is expanded and the nodes 3,8, and 13 are generated. Node 3 is immediately killed using the bounding function. Nodes 8 and 13 are added to the queue of live nodes.

Node 18 becomes the next E-node. nodes 19,24, and 29 are generated. Nodes 19 and 24 are killed as a result of the bounding functions. Node 29 is added to the queue of live nodes.

Now the E-node is node 34.Figure 8.1 shows the portion of the tree of Figure 7.2 that is generated by a FIFO branch-and-bound search. Nodes that are killed as a result of the bounding functions are a "B" under them.

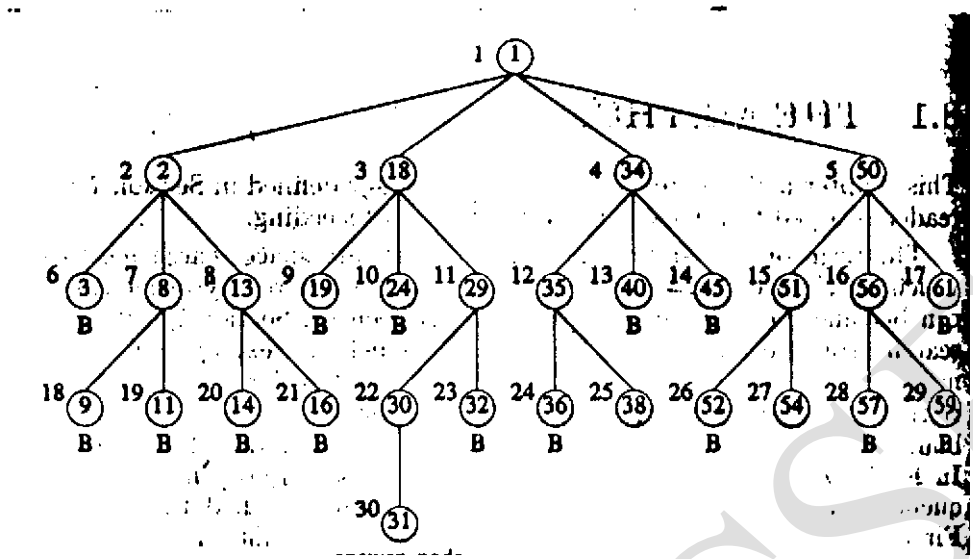
Numbers inside the nodes correspond to the numbers in Figure 7.2.Numbers outside the nodes give the order in which the nodes are generated by FIFO branch-and-bound.

At the time the answer node, node 31, is reached, the only live nodes remaining are nodes 38 and 54.

5.2. Least Cost (LC) Search:

- In LIFO Method, the selection (generation) of next E-node is a rigid one. The next E-node is selected in terms of level by level.
- The nodes in current level are expanded first and then go to the next level.
- This process is repeated until the nearest answer node is found
- In FIFO, it takes more time to search the nearest answer node, because of expansion of all nodes in each level. So we go for Least cost search(LC) method
- In LC search method, we can speed up the searching by ranking (cost) function.

MVIT-CSE



In Example 8.1, when node 30 is generated, it should have become obvious to the search algorithm that this node will lead to answer node in one move. However, the rigid FIFO rule first requires the expansion of all live nodes generated before node 30 was expanded.

The search for an answer node can often be speeded by using an "intelligent" ranking function $\hat{c}$ (.) for live nodes. The next E-node is selected on the basis of this ranking function.

. In the 4-queens example if we use a ranking function that assigns node 30 a better rank than all other live nodes, then node 30 will become E-node following node 29.The remaining live nodes will never become E-nodes as the expansion of node 30 results in the generation of an answer node (node 31).

The ideal way to assign ranks would be on the basis of the additional computational effort (or cost) needed to reach an answer node from the live node.For any node x, this cost could be

(1) The number of nodes on the sub-tree x that need to be generated before any answer node is generated or, more simply,

(2) The number of levels the nearest answer node (in the sub-tree x) is from x

Using cost measure (2), the cost of the root of the tree of Figure 8.1 is 4 (node 31 is four levels from node 1).The costs of nodes 18 and 34,29 and 35,and 30 and 38 are respectively 3, 2, and 1.The costs of all remaining nodes on levels 2, 3, and 4 are respectively greater than 3, 2, and 1.

Using these costs as a basis to select the next E-node, the E-nodes are nodes 1, 18, 29, and 30 (in that order).The only other nodes to get generated are nodes 2, 34, 50, 19, 24, 32, and 31.

The difficulty of using the ideal cost function is that computing the cost of a node usually involves a search of the sub-tree x for an answer node. Hence, by the time the cost of a node is determined, that sub-tree has been searched and there is no need to explore x again. For this

MVIT-CSE

reason, search algorithms usually rank nodes only on the basis of an estimate $\hat{g}$ (.) of their cost.

Let $\hat{g}$ (x) be an estimate of the additional effort needed to reach an answer node from x. node x is assigned a rank using a function (.) such that $\hat{c}$ (x)=f(h(x))+ $\hat{g}$ (x),where h(x) is the cost of reaching x from the root and f(.) is any non-decreasing function.

- **A search strategy that uses a cost function c'(x)=f(h(x)+g'(x)) to select the next E-node (current live node is chosen as next E-node) with least Cost. Hence such a strategy is called Least Cost (LC) search.**
  h(x) – cost of reaching x from root (no of levels form root to any node x)
  g(x)-cost of reaching a nearest answer node(answer node with minimum cost) from any node x(no of levels form any node x to a nearest answer node)
- Assign the rank (cost) to the node requires
  - Computation of cost needed to search an answer node from the live
  - For any node x, the cost is the number of levels the nearest answer node is from x.(that is number of levels from x to the nearest answer node)

Cost function c(.) is defined as,if x is an answer node ,then c(x) is the cost (level, computational difficulty,etc.) of reaching x from the root of the state space tree . If x is not an answer node, then c(x)=infinity, providing the sub-tree x contains no answer node; otherwise c(x) is equals the cost of a minimum cost answer node in the sub-tree x. It should be easy to see that $\hat{c}$ (.) with f(h(x))=h(x) is an approximation to c(.).

5.2.1. Control Abstractions for LC Search

Let t - state space tree
c( ) - cost function for the nodes in t

If x is a node in t, then c(x) is the minimum cost of any answer node in the subtree with root x. Thus , c(t) – cost of a minimum cost answer node in t.We can not find c(.) easily, so heuristic C'() is used.

```
listnode = record
{
listnode *next, *parent;
Float cost;
}

Algorithm LC Search(t)
\\ Search t for an answer node
{
        If *t is an answer node then output *t and return;
        E = t ; \\ E-node
        Initialize the list of live nodes to be empty;
        Repeat
        {
                For each child x of E do
```


MVIT-CSE

```
            {
                If x is an answer node the output the path
                    from x to t and return
                    Add(x) ;  \\ x  is  a new live node.
                    (x->parent):=E; \\Pointer for path to root.
            }
            If  there are no more live nodes then
            {
                Write("No answer  node"); return;
            }
            E=Least();
        } until (false);
}
```

- The LC Search algorithm uses c' to find an answer node
- Two function Add(x) and Least() are used
  - Add(x) -> Adds new live node x to the list of live nodes
  - Least() -> finds a live node with least c'(). This node is deleted form the list of live nodes and returned.
- It outputs the path from the answer node it finds to the root node t
  - If each x becomes live node , then associate a field parent which gives the parent of node x
  - When answer node g is found, the path from g to t can be determined.

    g - answer node
    x - live node
    t - root node

Algorithm explanation :-

1) Variable E point the current E-node
2) Initially root node t is the first E-node ie E = t ;
3) Initially the list of live nodes are empty
4) For loop examines all the children of the E-node
   a. If x (one of the children) is answer node then output the path from x to t and exit
   b. If x is not a answer node then x becomes a live node and add it to the list of live nodes. Its parent pointer is set to E
5) When all the children are generated, E-node becomes a dead node
6) If there are no more live nodes then print no answer node and exit
   else call Least() to choose the next E-node
7) repeat the steps 4 and 6 till if any answer node is found or if entire space tree has been searched

5.2.2. Example : 15 Puzzle Problem

MVIT-CSE

- The 15 puzzle consists of 15 numbered tiles on a square frame with a capacity of 16 tiles.
- Initial arrangement of the tiles are given, the objective of the problem is to transform this arrangement into the goal arrangement through a series of legal moves.
- The legal moves are ones in which a tile adjacent to the empty spot is moved to Empty Spot

Figure 8.5 –Problem State

- From the initial arrangement, four moves are possible
- We can move any one of the tiles numbered 2,3,5,6 to the empty spot
- These arrangements are called states of the puzzle
- To speed up the search, we can associate the cost c(x) with each node x in the state space tree.
- C'(x)=f(x)+g'(x) where f(x) – length of the path from root to node x

  g'(x) – an estimate of the length of a shortest path from x to a goal node in the subtree with root x

  (or)

  g'(x) – number of nonblank tiles not in their goal position

Figure 8.3- Part of the state space tree for the 15 puzzle

- An LC search start with root node (node 1) as E-node
- The E-node node 1 generates all its children 2,3, 4 and 5
- After E-node node 1 has generated all its children it becomes dead node
- The live nodes are 2,3 4 and 5
- To determine the next E-node, C'(x) is computed for all live nodes and the live node with least C'(x)is selected as next E-node

  C'(x)=f(x)+g'(x)

  C'(2)=1+4 = 5
  C'(3)=1+4 = 5
  C'(4)=1+2 = 3 (least cost node)
  C'(5)=1+4 = 5

  - The live node with least C' is node 4, which becomes the next E-node. It generates all its children 10,11 and becomes dead node
  - The live nodes are 2,3,5,10,11 and 12
- To determine the next E-node, C'(x) is computed for all live nodes and the live node with least C'(x)is selected as next E-node

  C'(x)=f(x)+g'(x)

  C'(2)=1+4 = 5
  C'(3)=1+4 = 5
  C'(5)=1+4 = 5
  C'(10)=2+1 = 3 (least cost node)

MVIT-CSE

C'(11)=2+3 = 5

- The live node with least C' is node10, which becomes the next E-node. It generates all its children 22 and 23 and becomes dead node
- The node 23 is determined as goal node and the search terminates.

In this case LC search is as efficient as using exact function C'().The LC search will be far more selective than any other search methods

5.3.Bounding

- A branch -and-bound searches the state space tree using any search mechanism in which all the children of the E-node are generated before another node becomes the E-node.

- We assume that each answer node x has a cost c(x) associated with it and that a minimum-cost answer node is to be found. Three common search strategies are FIFO, LIFO, and LC.

- A cost function $\hat{c}$(.) such that $\hat{c}$(x) <=c(x) is used to provide lower bounds on solutions obtainable from any node x.
- If **upper** is an upper bound, then all live nodes x with $\hat{c}$(x)>upper may be killed as all answer nodes reachable from x have cost c(x)>=c'(x)>upper6
- The starting value for upper can be set to infinity.
- Each time a new answer node is found, the value of upper can be updated
- C'(x) and upper and c(x) can be computed using formulas

5.3.1. Example: Job Sheduling with deadlines Problem

We are given n jobs and one processor. Each job i has associated with it a three tuple (P_i d_i t_i).job i requires t_i units of processing time .if it's processing is not completed by the deadline d_i, then a penalty P_i is incurred.

We are given n jobs and one processor. Each job i has associated with it a 3 tuples (P_i,d_i,t_i),where

t_i - processing time in units
d_i - deadline
P_i - penalty. If the job i is not completed within the deadline **di**, then penalty P_i is incurred

Objective of the problem:

To select the subset J of n jobs such that all jobs in J can be completed by their deadlines. Hence a penalty is incurred only on those jobs not in J. The subset J should have minimum penalty among all possible subsets J. such that a J is optimal.

Let us assume that n=4
(P1,d1,t1) = (5,1,1)
(P2,d2,t2) = (10,3,2)
(P3,d3,t3) = (6,2,1)
(P4,d5,t4) = (3,1,1)

- The solution space for this instance consist of all possible subsets of the job index set {1,2,3,4 }

  job index : 1,2,3,4
  solution space : all possible subset of job index
  ie {1,2,3} {2,3} {1,4} {1,3,4} ,……..

- The solution space can be organized into a tree by means of two formulations
  1. variable tuple size formulation
  2. fixes tuple size formulation

1.Variable Tuple Size Formulation

node – answer node

node - infeasible subsets

Compute c(x) using the following rule:

1. if x is a leaf node,

c(x) – summation of penalty of the jobs which are not in the path from root node to any node x

2. if x is a root node

c(x) - minimum penalty corresponding to any node in the subtree within root x

(or)

c(x)=min{c(child-1),c(child-2),….c(child-n) ,where x is ode in subtree

c(x) - $\propto$ for node

MVIT-CSE

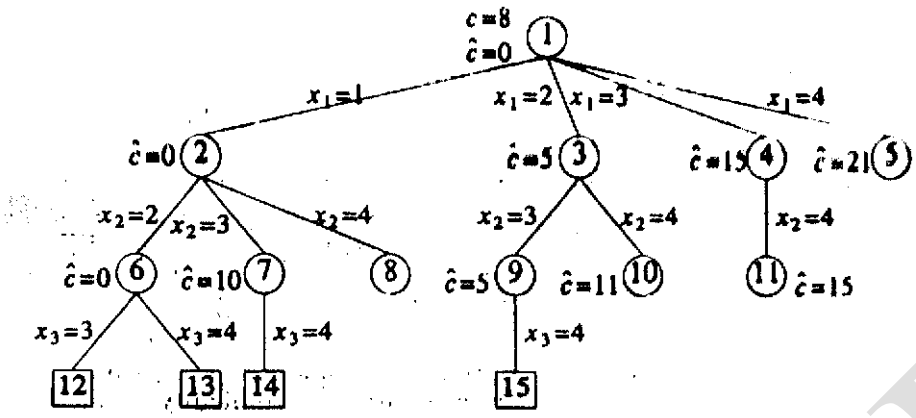
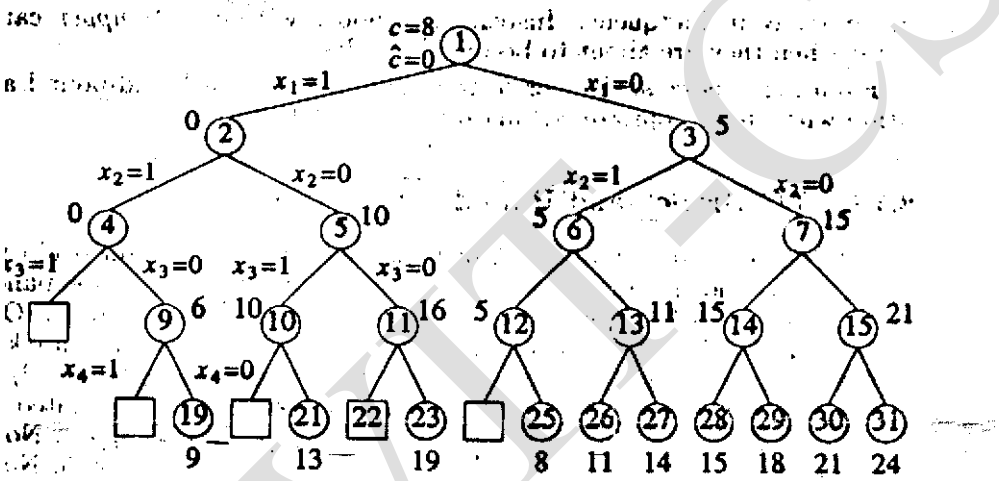


Figure 8.6 State space tree corresponding to variable tuple size formulation



ure 8.7 State space tree corresponding to fixed tuple size formulation

Figure 8.6 corresponds to the variable tuple size formulations while figure 8.7 corresponds to the fixed tuple size formulation. In both figures square nodes represent infeasible subsets. In figure 8.6 all non-square nodes are answer nodes. Node 9 represents an optimal solution and is the only minimum-cost answer node .For this node j={2,3} and the penalty (cost) is 8. In figure 8.7 only non-square leaf nodes are answer nodes. Node 25 represents the optimal solution and is also a minimum-cost answer node. This node corresponds to j={2,3} and a penalty of 8. The costs of the answer nodes of figure 8.7 are given below the nodes.

5.2. TRAVELLING SALESMAN PROBLEM

INTRODUCTION :

It is algorithmic procedure similar to backtracking in which a new branch is chosen and is there (bound there) till new branch is choosing for advancing.

MVIT-CSE

This technique is implemented in the traveling salesman problem [TSP] which are asymmetric (Cij <>Cij) where this technique is an effective procedure.

- Let G(V,E) be a directed graph defining an instance of the traveling salesman problem.
- Let Cij be the cost of edge<i,j>.Cij=infinity if<i,j> is not in E.
- Let $|v|=n$.
- Without loss of generality we can assume that every tour starts and ends at vertex 1.
- So solution space is given by S={1, Π,1}
  Π is a permutation of {2,3,…n}
  Then $|S| = (n-1)!$
- The size of S can be reduced by restricting S .So that $(1,i_1,i_2\ldots i_{n-1},1) \in S$ iff $(i_j,i_{j+1}) \in E$, $0 \le j \le n-1$ and $i_0=i_n=1$.
- In TSP Cij<>Cji

Procedure :

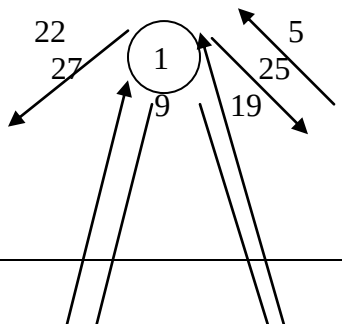
Step 0: Generate cost matrix C for the given graph G
Step 1: ROW REDUCTION: For all rows do step 2
Step 2: Find least cost in a row and negate it with rest of the elements.
Step 3: COLUMN REDUCTION:Use cost matrix Row reduced one.For all columns do Step 4.
Step 4: Find least cost in a column and negate it with rest of the elements.
Step 5: Preserve cost matrix C [in which row reduced first and then column reduced next] for the i th time.
Step 6: Enlist all edges (i, j) having cost = 0.
Step 7: Calculate effective cost of the edges.

$\sum C_{ij}$ = least cost in the ith row excluding (i, j) + least cost in the j th column excluding (i, j).

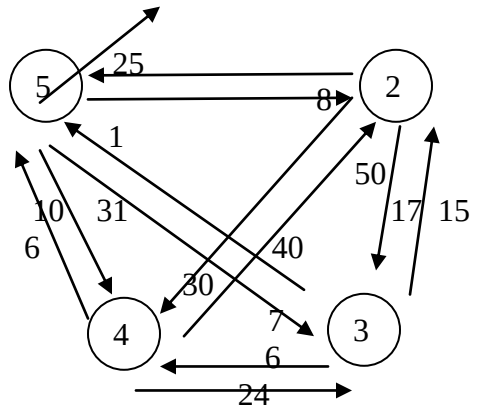
Step 8: Compare all effective cost and pick up the largest l(EC). If two or more have the same cost then arbitarily choose any one among them.

Step9: Delete (i, j) : delete ith row and jth column.Change (j, i) value to infinity.(to avoid infinite loop formation). If (i,j) not present,leave it.
Step 10: Repeat step 1 to step 9 until the resultant cost matrix having order of 2*2 and reduce it. (Both R.R and C.C)
Step 11: Use preserved cost matrix Cn, Cn-1, ……,C1.Choose an edge [i, j] having value =0 at the first time from a preserved matrix and leave that matrix.
Step 12:Use result obtained in Step 11 to generate a complete tour.

Example: Given graph G



MVIT-CSE



Step 0:Generate Cost Matrix

	1	2	3	4	5
1	α	25	40	31	27
2	5	α	17	30	25
3	19	15	α	6	1
4	9	50	24	α	6
5	22	8	7	10	α

Phase 1:

Step1: Row Reduction in C.

In R1,an edge <1,2> = 25
In R2,an edge <2,1> = 5
In R3,an edge <3,5> = 1
In R4,an edge <4,5> = 6
In R5,an edge <5,3> = 7
Subtract these elements from other elements.

1 2 3 4 5

MVIT-CSE

1	α	0	15	6	2
2	0	α	12	25	20
3	18	14	α	5	0
4	3	44	18	α	0
5	15	1	0	3	α

Step 3: Column Reduction in C1

In C1,an edge <2,1> = 0
In C2,an edge <1,2> = 0
In C3,an edge <5,3> = 0
In C4,an edge <5,4> = 3
In C5,an edge <3,5>&<4,5> = 7

Subtract these elements from other elements.

	1	2	3	4	5
1	α	0	15	3	2
2	0	α	12	22	20
3	18	14	α	2	0
4	3	44	18	α	0
5	15	1	0	0	α

Step 5:

Preserve the above in C1,

	1	2	3	4	5
1	α	0	15	3	2
2	0	α	12	22	20
3	18	14	α	2	0
4	3	44	18	α	0
5	15	1	0	0	α

MVIT-CSE

Step 6: List all edges having cost =0.

L= $\{(1,2),(2,1),(3,5),(4,5),(5,3),(5,4)\}$

Step 7: Calculate effective cost of edges [E.C]

EC for <1,2> = 2+1 =3
EC for <2,1> = 12+3 = 15
EC for <3,5> = 2+0 =2
EC for <4,5> = 3+0 = 3
EC for <5,3> = 0+12 = 12
EC for <5,4> = 0+2 = 2

Step 8 : EC (l) <2,1>is having the largest length or highest cost.

Step9 : Delete (2,1) from C1 .Change (1,2) → α if exists.

i.e: remove 2nd row and 1st column and <1,2>=infinity.

Now Cost Matrix =

	2	3	4	5
1	α	15	3	2
3	14	α	2	0
4	44	18	α	0
5	1	0	0	α

Step 10: The order of reduced Cost matrix ≠ 2 x 2 .Therefore goto step 1.

Phase 2:

Step 1 : Do RR

	2	3	4	5
1	α	13	1	0
3	14	α	2	0
4	44	18	α	0
5	1	0	0	α

Step 3: Do CR

2 3 4 5

MVIT-CSE

1	α	13	1	0
3	13	α	2	0
4	43	18	α	0
5	0	0	0	α

Step 5: Preserve the above in C2

C2 =

	2	3	4	5
1	α	13	1	0
3	13	α	2	0
4	43	18	α	0
5	0	0	0	α

Step 6: L={(1,5), (3,5), (4,5), (5,2), (5,3), (5,4)}
Step 7: calculation of E.C.

EC of edge (1,5) = 1+0 =1
EC of edge(3,5) = 2+0 =2
EC of edge (4,5) = 18+0 =18
EC of edge (5,2) = 0+13 =13
EC of edge (5,3) = 0+13 =13
EC of edge (5,4) = 0+1 =1

Step 8: L having an edge (4,5) is the largest.
Step 9: Delete (4,5) from C2 and make change in it as (5,4) = α if exists.
Now, cost matrix =

	2	3	4
1	α	13	1
3	13	α	2
5	0	0	α

Step 10: The order of cost matrix ≠ 2x2 hence goto step 1
Phase 3:
Step 1: Do RR

2 3 4

MIT-CSE

1	α	12	0
3	11	α	0
5	0	0	α

Step 3: Do CR

	2	3	4
1	α	12	0
3	11	α	0
5	0	0	α

Step 5: Preserve the above in C3

	2	3	4
1	α	12	0
	11	α	0
	0	0	α

Step 6: L={(1,4), (3,4), (5,2), (5,3)}

Step 7: Calculation of E.C

EC of edge (1,4)=12+0=12

EC of edge (3,4)=11+0=11

EC of edge (5,2)=0+11=11

EC of edge (5,3)=0+12=12

Step 8: Here we are having two edges (1,4) and (5,3) with cost = 12. Hence arbitrarily choose (1,4)

Step 9: Delete (1,4).Change (4,1) = α if exists.

Now cost matrix is

	2	3
2	11	α
3	0	0

Step 10: The order of the reduced cost matrix = 2x2

Do RR:

	2	3
3	0	α
5	0	0

Do CR:

	2	3

MVIT-CSE

3	0	α
5	0	0

Therefore C4 =

	2	3
3	0	α
5	0	0

Step 11: List C1, C2, C3 & C4

C4 :

	2	3
3	0	α
5	0	0

C3:

	2	3	4
1	α	12	0
3	11	α	0
5	0	0	α

C2 :

	2	3	4	5
1	α	13	1	0
3	13	α	2	0
4	43	18	α	0
5	0	0	0	α

C1:

1 2 3 4 5

MVIT-CSE

1	α	0	15	3	2
2	0	α	12	22	20
3	18	14	α	2	0
4	3	44	18	α	0
5	15	1	0	0	α

Step 12:

Use C4 =

	2	3
3	0	α
5	0	0

Pick up an edge (i, j)=0 having least index. Here (3,2) =0
Hence, T← (3,2)

Use C3 =

	2	3	4
1	α	12	0
3	11	α	0
5	0	0	α

Pick up an edge (i, j)=0 having least index.Here (1,4) =0
Hence, T←(3,2), (1,4)

Use C2=

	2	3	4	5
1	α	13	1	0
3	13	α	2	0
4	43	18	α	0
5	0	0	0	α

Pick up an edge (i, j) with least cost index. Here (1,5) → not possible. B'coz city1 has already choosen. Then (3,5) → not possible as already chosen index.
Next choose (4,5) →0
Hence, T← (3,2), (1,4), (4,5)
Use C1 =

1	2	3	4	5

MVIT-CSE

1	α	0	15	3	2
2	0	α	12	22	20
3	18	14	α	2	0
4	3	44	18	α	0
5	15	1	0	0	α

Pick up an edge (i, j) with least index.
(1,2) → Not possible (2,1) → Choose it
Hence T← (3,2), (1,4), (4,5), (2,1)

Solution:
From the above list

3—>2—>1—>4—>5

Now this results , Travelling Salesperson started his tour from city3 and visited city2,city1,city4 ,city 5 and returned to the starting city3.
Final result:

3—>2—>1—>4—>5—>3

Cost is 15+15+31+6+7=64

5.4. 0/1 Knapsack Problem

- To use branch and bound technique to solve any problem, it is first necessary to conceive of a state space tree for the problem.
- Branch and bound technique is used to solve minimization problem(eg 15 puzzle, 4queen, node with minimum cost is selection criteria)
- The branch and bound technique can not be applied directly in the knapsack problem , because the knapsack problem is a maximization problem(eg maximum profit is the selection criteria)
- This difficulty is easily overcome by replacing the objective function ∑PiXi by the function -∑PiXi
- The modified knapsack problem is stated as

$$\text{Minimize} \quad -\sum_{i=1}^{n} pixi$$

$$\text{subject to} \quad \sum_{i=1}^{n} wixi <= m$$

$$xi=0 \text{ or } 1, \quad 1<=I<=n$$

There are two types of formulation.
1.Fixed tuple size formulation
2.Variable tuple size formulation

Fixed tuple size formulation

- Every leaf node in the state space tree representing an assignment for which

$$\sum_{1<=i<=n} wixi <= m \text{ is an answer node}$$

MVJ-CSE

- All other leaf nodes are infeasible
- For minimum cost answer node to correspond to any optimul solution, we need to define $c(x) = -\sum_{1<=i<=n} pixi$ for every answer node x
- The cost $c(x) = \alpha$ for infeasible leaf nodes

Example:

Consider the knapsack instance n=4 , (p1,p2,p3,p4)=(10,20,12,18), (w1,w2,w3,w4) =(2,4,6,9), and m=15

- There are two methods to solve the knapsack problem
  - LC Branch and Bound Solution
  - FIFO Branch and Bound Solution

5.4.1. LC Branch and Bound

The search begins with root as the E-node. That is node1(root) becomes E-node.

Algorithm:

Algorithm Ubound(cp,cw,k,m)

// cp is the current profit total ,cw is the current weight total; k is the index of the last removed item; and m is the knapsack size//

```
{
        b=cp; c=cw;
        for i=k+1 to n do
        {
                if((c+w[i])<=m) then
                {
                        c=c+w[i]; b=b-p[i];
                }
        }
        return b;
}
```

Algorithm Explanation with Example:

Ubound(0,0,0,15)

b=0; c=0;

for i= 1 to 4

i=1;

if ((c+w[1])<=m} (c=0 and w[1]=2 and 0+2=>2<=15) **true**

c=0+2= 2

b=0-10= -10

i=2

if ((c+w[2])<=m} (c=2 and w[2]=4 and 2+4=6<=15) **true**

c=2+4=6

b=-10-10= -20

i=3

if ((6+w[3])<=m} (c=6 and w[2]=6 and 6+6=12<=15) **true**

c=6+6=12

b=-20-12= -32

i=4

if ((15+w[4])<=m} (c=12 and w[2]=9 and 12+9=21<=15) **False**

MVIT-CSE

return b(b=-32); (root node)=b

- For any node x in the state space tree, we compute c'(x) and u(x) and upper
- For any node x , if c'(x)>upper the node x is killed immediately
- Initially the upper =u(1) and then the upper is updated by the node with minimum u(x) value in current level.
- u() for root node is calculated using the algorithm Ubound or (-∑ fully included objects .
- u(1) =b(return value of UBound algorithm)
  =-32 (objects1, 2 and 3 are fully includes so -∑ p1+p2+p3
  =-(10+10+12)=-32)

Node1:

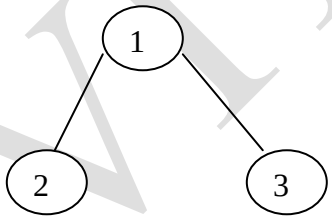
Initially, first 3 whole objects and part of 4 th object are included in the knapsack, that is (1,1,1,3/9)

$c'(x) = -\sum_{1<=i<=n} pixi$

c'(1)= - 1*10+1*10+1*12+3/9*18
= -32

upper =u(1)
=-32(-∑ fully(not a part of object) included objects (that is1, 2 and 3) -∑p1+ p2+p3
=-(10+10+12)=-32)

- E-node node 1 generates all its children 2,3 and becomes dead node. Now node 2 and 3 are live nodes.
- To determine the next E-node compute c'(x),u(x) and upper for live nodes 2 and 3



Node 2:

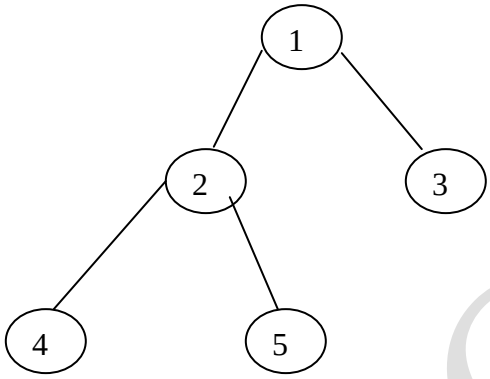
- If first object is included (already the first object is included in node 1 so same value is obtained for c'(2))
- Status of Included objects; (1,1,1,3/9)
- c'(2)= - 1*10+1*10+1*12+3/9*18
  = -38
- u(1)=-32(-∑ fully included objects (tat is1, 2 and 3) -∑p1+ p2+p3 =-(10+10+12)=-32)

Node 3:

- If first object is not included ,
- Status of Included(objects; (0,1,1,5/9)
- c'(3)= - 0*10+1*10+1*12+5/9*18
  = -32

MVIT-CSE

- u(3)=-22(-∑ fully included objects (tat is 2 and 3) -∑ p2+p3 =-(10+12)=-22)
- upper=min{u(2),u(3)}
  =min{-32,-22}
  upper=-32
- if c'(2) and c'(3) < upper, so node 2 and 3 are not killed and are live nodes
- The node 3 becomes the next E-node because c'(2)=-38<c'(3)=-32
- Node 3 generates all its children 4, 5

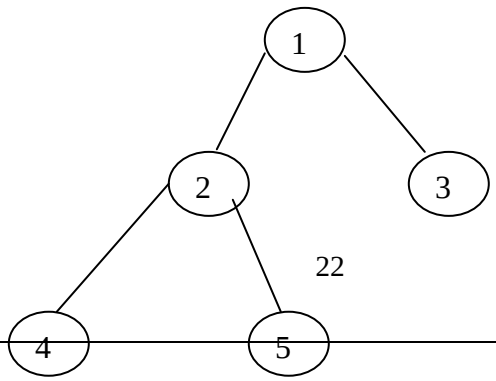


Node 4:

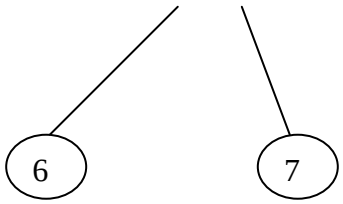
- If second object is included (already the third object is included in node 1,2 so same value is obtained for c'(4))
- status of Included(objects; (1,1,1,3/9)
- c'(4)= - 1*10+1*10+1*12+3/9*18
  = -38
- u(4)=-32(-∑ fully included objects (tat is1, 2 and 3) -∑p1+ p2+p3 =-(10+10+12)=-32)

Node 5:

- If second object is not included ,
- status of Included objects; (1,,0,1,7/9)
- c'(5)= - 1*10+0*10+1*12+7/9*18
  = -36
- u(5)=-22(-∑ fully included objects (that is 1 and 3) -∑ p1+p3 =-(10+12)=-22)
- upper=min{u(4),u(5)}
  =min{-32,-22}
  upper=-32
- if c'(4) and c'(5) < upper, so node 4 and 5 are not killed and are live nodes
- Within the live nodes 4 and 5,the node 4 becomes the next E-node because c'(4)=-38<c'(5)=-36
- Node 4 generates all its children 6,7



MVJ-CSE

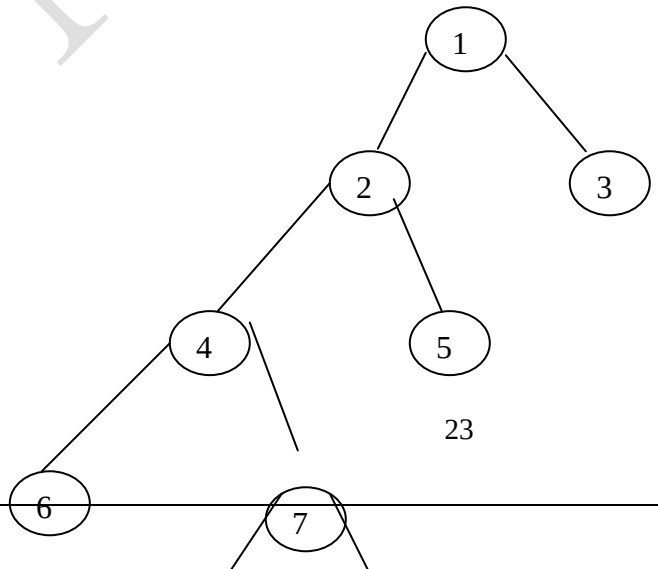


Node 6:

- If third object is included (already the third object is included in node 1,2,4,so same value is obtained for c'(6))
- Status of Included objects; (1,1,1,3/9)
- c'(6)= - 1*10+1*10+1*12+3/9*18
  = -38
- u(6)=-32(-∑ fully included objects (tat is1, 2 and 3) -∑p1+ p2+p3 =-(10+10+12)=-32)

Node 7:

- If third object is not included ,
- Status of Included objects; (1,1,0,1)
- c'(7)= - 1*10+1*10+0*12+1*18
  = -38
- u(7)=-38(-∑ fully included objects (tat is 1,2 and 4) -∑ p1+p2 +p4=-(10+10+18)=-38)
- upper=min{u(6),u(7)}
  =min{-32,-38}
  upper=-38
- if c'(6) and c'(7) = upper, so node 4 and 5 are not killed and are live nodes
- Within the live nodes 6 and 7,the node 7 becomes the next E-node because u(7)=-38<u(6)=-32 (c'(6)=-38 is equal to c'(7)=-36 so compare u(x)))
- Node 7 generates all its children 8,9



MVJT-CSE

Node 8:

- If fourth object is included
- Status of Included objects; (1,1,0,1)
- c'(6) = - 1*10+1*10+0*12+1*18
  = -38
- u(8)=-38(-∑ fully included objects (tat is1, 2 and 4) -∑p1+ p2+p4
  =-(10+10+18)=-38)

Node 9:

- If fourth object is not included ,
- status of Included objects; (1,1,0,0)
- c'(7)= - 1*10+1*10+0*12+0*18
  = -20
  - u(7)=-20(-∑ fully included objects (tat is 1 and 2) -∑ p1+p2 =-(10+10)=-20)
  - upper=min{u(8),u(9)}
    =min{-38,-20}
    upper=-38
  - if c'(9)=-20 > upper, so node and 9 is killed
  - Nodes 6 and 8 are live nodes with least costs . Regardless of which becomes the next
    E-node ,c'(E)>=upper and the search terminates with node 8, the answer node
  - At this time the value of upper –38 together with the path 8,7,4,2,1 is printed
    As output

5.4.2. FIFO Branch and Bound

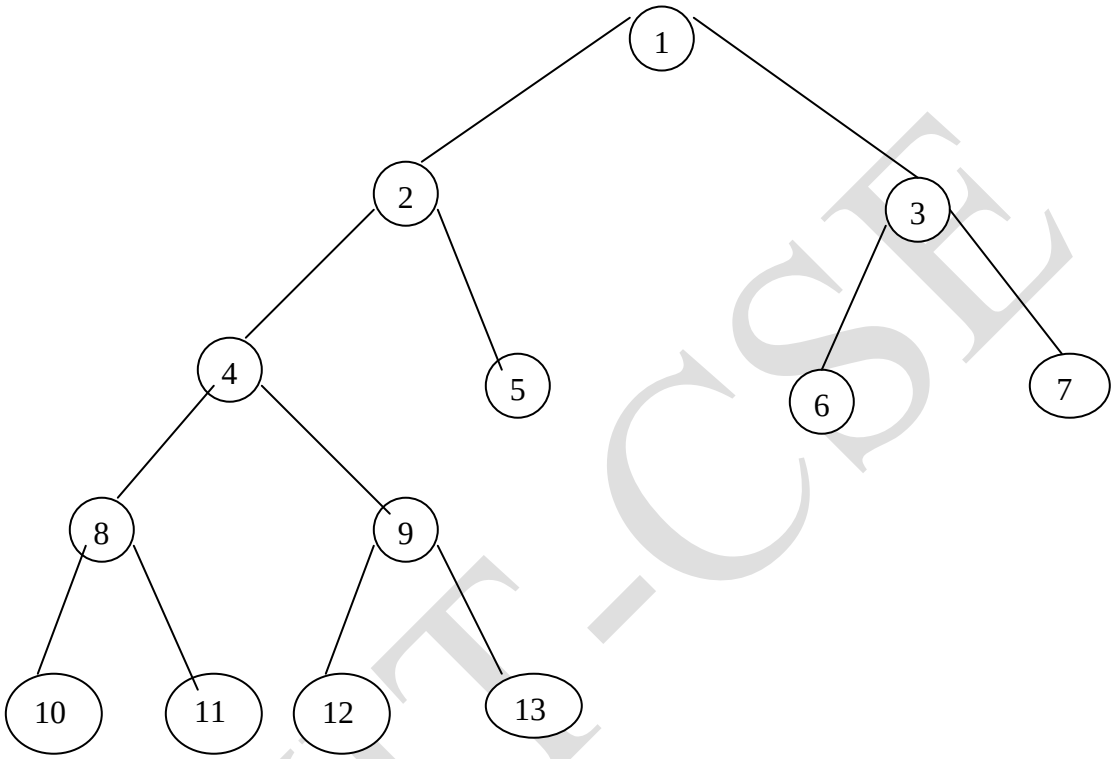
- for any node x in the state space tree, we compute c'(x) and u(x) and upper
- for any node x , if c'(x)>upper the node x is killed immediately
- initially the upper =u(1) and then the upper is updated by the node with minimum u(x) value in current level

u() for root node is calculated using the algorithm Ubound or (-∑ fully included objects u(1)=b(return value of UBound algorithm)=-32 (objects1, 2 and 3 are fully includes so -∑ p1+p2+p3 =-(10+10+12)=-32)

- Initially the root node, node1 is the E-node and queue of live nodes are empty
- Since this is not a solution node, upper is initialize to u(1)

MVIT-CSE

- Assume that, the children of a node are generated left to right. Node 2 and 3 are generated and added to the queue



- The c'(2) ,c'(3)and u(2),u(3) are calculated
- upper=min(u(2),u(3) that is upper =-32
- Node 2 becomes next E-node. Its children, nodes 4 and 5, are generated and added to the queue
- The c'(4),c'(5) and u(4),u(5) are calculated
- upper=min(u(4),u(5) that is upper =-32
- Node 3 becomes next E-node. Its children, nodes 6 and 7, are generated and added to the queue
- The c'(6),c'(7) and u(6),u(7) are calculated
- Node 7 is immediately killed because c'(7)>upper(-28>-32)
- upper=min(u(6),u(7) that is upper =-32
- Node 4 becomes next E-node. Its children, nodes 8 and 9, are generated and added to the queue
- The c'(8) ,c'(9)and u(8) ,u(9) are calculated
- upper=min(u(8),u(9) that is upper =-38

MVIT-CSE

- nodes 5 and 6 in the queue are the next E-nodes which are not expanded because c'(5) and c'(6) >upper(-36 and –32 >-38)
- Node 8 becomes next E-node. Its children, nodes 10 and 11, are generated and added to the queue
- The c'(10) ,c'(11)and u(10) ,u(11) are calculated
- Node 10 is infeasible so it is killed. Node 11 has c'(11)>upper so it is killed
- Node 9 becomes next E-node. Its children, nodes 12 and 13, are generated and added to the queue
- The c'(12) ,c'(13)and u(12) ,u(13) are calculated
- upper=min(u(12),u(13))that is upper =-38
- node 13 is killed because c'(13)>upper
- The only remaining node is 12. it has no children and search terminates
- The value of upper and the path from node 12 to toot is printed as output.

MVIT-CSE